Architecture Documentation

2026/11/05

Contents

1 Overview **3**

2 System Architecture **4**

3 Architectural Layers **5**
- 3.1 Local System Layer . . . 5
- 3.2 Core Application Layer (`aspera-cli` gem) . . . 5
  - 3.2.1 Entry Point . . . 5
  - 3.2.2 Runner and Context . . . 6
  - 3.2.3 CLI Options . . . 6
  - 3.2.4 Plugin System . . . 7
  - 3.2.5 Command DSL . . . 7
  - 3.2.6 Action style convention . . . 11
  - 3.2.7 Entity identifier placement . . . 12
  - 3.2.8 Mounting another plugin's commands . . . 13
  - 3.2.9 Legitimate residual imperative reads . . . 14
  - 3.2.10 Transfer Agent Abstraction . . . 14
- 3.3 API Communication Layer . . . 15
  - 3.3.1 REST Client . . . 15
  - 3.3.2 Product API Clients . . . 16
  - 3.3.3 API Schemas . . . 16
  - 3.3.4 OAuth Implementation . . . 16
- 3.4 FASP Transfer Layer . . . 16
  - 3.4.1 ASCP Installation Manager . . . 16
  - 3.4.2 Transfer Specification . . . 17
  - 3.4.3 Async Sync . . . 17
- 3.5 Remote Systems Layer . . . 17
  - 3.5.1 Web Applications (HTTPS) . . . 17
  - 3.5.2 Transfer Servers (FASP Protocol) . . . 17
  - 3.5.3 Third-Party Integrations . . . 18

4 Data Flow **19**
- 4.1 Typical Command Execution Flow . . . 19

5 Key Design Patterns **20**
- 5.1 Plugin Architecture . . . 20
- 5.2 Factory Pattern . . . 20
- 5.3 Singleton Pattern . . . 20
- 5.4 Strategy Pattern . . . 20
- 5.5 Command DSL Pattern . . . 20
- 5.6 Mixin / Module Pattern . . . 21

6 Configuration Management **22**
- 6.1 Configuration File . . . 22
- 6.2 Preset System . . . 22
- 6.3 Secret Management . . . 22

7 Error Handling **24**

7.1 Error Hierarchy . . . 24
7.2 Error Analysis . . . 24

8 Logging and Debugging **25**
8.1 Log Levels . . . 25
8.2 Debug Features . . . 25

9 Testing Architecture **26**
9.1 Test Structure . . . 26
9.2 CI/CD Integration . . . 26

10 Extension Points **27**
10.1 Adding a New Plugin . . . 27
10.2 Adding a New Transfer Agent . . . 27
10.3 Adding a New Output Format . . . 27

11 Performance Considerations **28**
11.1 Transfer Optimization . . . 28
11.2 API Optimization . . . 28

12 Security Architecture **29**
12.1 Authentication Methods . . . 29
12.2 Credential Storage . . . 29
12.3 Secure Communication . . . 29

13 Deployment Models **30**
13.1 Installation Methods . . . 30
13.2 Runtime Requirements . . . 30

14 References **31**

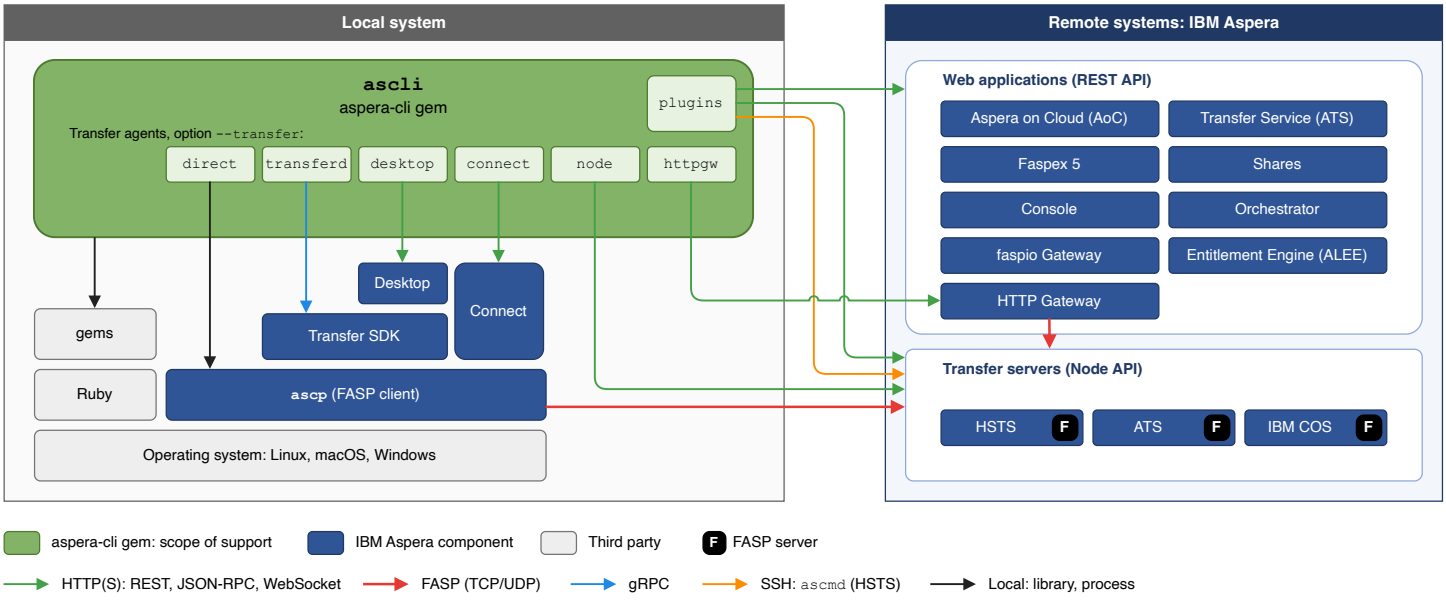
Chapter 1

Overview

The IBM Aspera CLI (`ascli`) is a Ruby-based command-line interface that provides unified access to IBM Aspera's high-speed file transfer products and services. The architecture follows a modular, plugin-based design that separates concerns between command processing, API communication, and transfer execution.

Chapter 2

System Architecture



Architecture Diagram

The architecture diagram illustrates the layered structure of `ascli` and its interactions with external components. Source: `architecture.drawio`, exported by `rake doc:diagrams`.

Chapter 3

Architectural Layers

3.1 Local System Layer

The foundation layer consists of the local execution environment:

- **Operating System**: Cross-platform support (Linux, macOS, Windows)
- **Ruby Runtime**: Ruby interpreter, minimum version `RUBY_MINIMUM_VERSION` in `lib/aspera/cli/info.rb` (CI tests: matrix in `.github/workflows/test.yml`)
- **Ruby Gems**: Third-party dependencies managed via Bundler
- **Transfer Agents**: Multiple FASP client implementations, selected with option `--transfer`
  - `ascp` (client): The core FASP protocol implementation (`direct`)
  - Transfer SDK (trSDK): gRPC-based transfer daemon (`transferd`)
  - Connect: Browser-based transfer client (`connect`)
  - Desktop: Aspera Desktop Client integration, over JSON-RPC; the client runs the Transfer SDK underneath (`desktop`)
- **`async`** (Aspera Sync): FASP client started by the `sync` commands, see Async Sync
- **Remote Transfer Agents**: The transfer runs on a server, see Remote Systems Layer
  - HTTPGW: HTTP Gateway for firewall-friendly transfers (`httpgw`)
  - Node: Direct Node API transfers (`node`)

3.2 Core Application Layer (`aspera-cli` gem)

The central green component in the diagram represents the Ruby gem that implements all CLI functionality.

3.2.1 Entry Point

File: `bin/ascli`

The main executable script that:

- Sets up UTF-8 encoding for internationalization
- Pre-parses early logging options before full initialization: `--log-level` / `--log.level`, `--log-format` / `--log.format`, `--logger` / `--log.type`
- Loads optional code coverage (`aspera/coverage`)
- Fixes the home directory on Windows via `Environment.instance.fix_home`
- Delegates to the main CLI processor

```
#!/usr/bin/env ruby
# Pre-parses logging options before full initialization
ARGV.each { |arg| ... }
require 'aspera/coverage'
require 'aspera/environment'
require 'aspera/cli/runner'
Aspera::Environment.instance.fix_home
Aspera::Cli::Runner.new(ARGV).run
```

3.2.2 Runner and Context

Files: lib/aspera/cli/runner.rb, lib/aspera/cli/context.rb

The Runner class orchestrates the full command lifecycle:

- **run**: Main entry point — calls run_with_result, displays the result via Formatter, handles all exceptions, and exits with the appropriate status code.
- **run_with_result**: Pure computation entry point — initializes all agents and options, resolves the target plugin, executes the action, and returns a Result object. Raises on error. Used by the MCP server (mcp/tool.rb) to run commands in-process.

All shared objects are held in a Context instance and passed to plugins by reference. Members: options (Parser), transfer (TransferAgent), config (Plugins::Config), formatter, persistency, man_header, presets (PresetManager), vault (VaultManager), http_config (Http), main_folder, mailer (Mailer), secret_finder, plus progress_bar, pac_executor and help_requested.

Runner#init_agents_and_options builds the context in this order:

1. Formatter and Parser, then global options
2. Bootstrapper (run): one-time setup of shared services, before any plugin is instantiated:
   - main_folder (option home), persistency, presets (config file), vault (options vault, vault_password), http_config, progress_bar
   - plugin lookup folders
   - @preset: and @vault: extended-value handlers
   - global singletons: Rest::Parameters, OAuth::Factory, SSL, Transfer::Parameters, Rest::ErrorAnalyzer
   - PAC proxy executor (option fpac)
3. Plugins::Config (only a CLI plugin: option declaration and command handlers), then Mailer, SecretFinder and TransferAgent

Context#validate checks that all mandatory members are set.

3.2.3 CLI Options

Files:

- parser.rb — Cli::Parser: command-line argument processing, get_next_command, get_next_argument, instance_identifier
- option_declarator.rb, option_registry.rb, option_value.rb, option_types.rb — option declaration, storage and type validation
- extended_value.rb — Extended Value Syntax (@json:, @yaml:, @ruby:, @preset:, @vault:, @args:, ...)
- options.schema.yaml — schema for global options

Key responsibilities:

- Declare and validate CLI options (boolean, string, integer, array, hash types)
- Composite options with dot-notation sub-keys (e.g. --out.format=json, --log.level=debug)
- Handle sensitive data (passwords, secrets) with masking (secret/hider.rb)
- Provide option inheritance and defaults from presets

Options are declared with the option DSL of OptionDeclarator, not only by plugins: Formatter, TransferAgent, Http and Wizard extend OptionDeclarator (plugins include it through Base). Formatter and TransferAgent declare their options on the parser with declare_options(parser, target: self), so that on_set: callbacks are methods of that instance. Config includes the options of Wizard and Http with use_options (for Http, target: gives the object of the on_set: callbacks: Context#http_config).

Global and bootstrap options (Runner#declare_global_options, Bootstrapper, Parser#initialize) are intentionally declared with options.declare, not with the option DSL: their defaults are read at runtime (current state of Log, TempFileManager, Secret::Hider and Environment singletons, config_file under the home folder, home after Environment#fix_home), and each bootstrap phase parses its options right before using them.

3.2.4 Plugin System

Directory: `lib/aspera/cli/plugins/`

The plugin architecture enables modular command implementation for different Aspera products:

Base Plugin (`base.rb`):

- Provides the Command DSL (`command`, `commands_under`, `crud_commands`, `option`, ...) and the dispatcher
- Provides standard CRUD operations: `create`, `list`, `modify`, `show`, `delete` (`Base::Operations`)
- Provides bulk operation support (`--bulk`, `--bfail`)
- Implements resource identifier resolution (including percent-selector syntax)
- Manages plugin context (options, transfer agent, config, formatter)

Product Plugins (registered and exposed as top-level commands):

- `aoc.rb` - Aspera on Cloud
- `ats.rb` - Aspera Transfer Service
- `faspex5.rb` - Faspex 5
- `shares.rb` - Aspera Shares
- `node.rb` - Node API
- `console.rb` - Aspera Console
- `orchestrator.rb` - Aspera Orchestrator
- `server.rb` - HSTS (High-Speed Transfer Server)
- `cos.rb` - IBM Cloud Object Storage
- `httpgw.rb` - HTTP Gateway
- `faspio.rb` - Fasp.io Gateway
- `alee.rb` - Aspera Line Enterprise Edition

The Faspex 4 plugin was removed (end of support).

Utility Plugins (registered commands but not product-specific):

- `config.rb` - Configuration management (includes `SyncActions`, `VaultActions`, `GemChecker`, `AscpActions`, `PresetActions`, `TransferActions` mixins)
- `preview.rb` - File preview generation
- `mcp.rb` - Model Context Protocol server (exposes `ascli` to AI assistants)

Internal Base Classes (excluded from factory registration via `IGNORE_PLUGINS`):

- `base.rb` - Abstract base class for all plugins
- `basic_auth.rb` - Abstract base class for plugins using basic authentication (url/username/password)
- `oauth.rb` - Abstract base class for plugins using OAuth
- `factory.rb` - Plugin factory (singleton); discovers plugins by scanning the plugin folders

3.2.5 Command DSL

All plugins declare their command tree using a class-level DSL defined in `Base`, which makes the full command tree statically introspectable, self-documenting, and testable without execution.

Key files:

- `lib/aspera/cli/command_spec.rb` — data classes: `CommandSpec`, `ArgumentSpec`, `OptionSpec`
- `lib/aspera/cli/command_registry.rb` — flat registry keyed by full path (`Array<Symbol>`), with `validate!`
- `lib/aspera/cli/plugins/base.rb` — DSL class methods and dispatcher

DSL class methods (in `Base`):

Method	Purpose
`command(id, **kwargs)`	Register a `CommandSpec`; `parent:` defaults to the enclosing `commands_under` scope

Method	Purpose
`commands_under(parent, description: nil) { … }`	Scope block setting the default parent for nested `command` calls. Re-entrant; `parent` is relative to the current scope. Auto-declares the parent node (`"Manage <name>"`) if not yet declared
`crud_commands(api:, entity:, operations:, name:, lookup:, **kwargs)`	Declare one leaf command per CRUD verb for a REST entity (see below)
`define_action_method(path) { … }`	`define_method` with the conventional `action_<path>` name; used for homogeneous generated commands
`option(name, description:, short:, allowed:, default:, on_set:, shorthand:, deprecation:, schema:)`	Declare a plugin option (stored as `OptionSpec`, declared on the parser in `Base#initialize`). Raises if an ancestor already declares it. The option always stores its value (read with `get_option`, shown by `--show-config`, merged for `Hash`/`Array`). `on_set:` is only needed to push the value to another object: called with the new (merged) value each time it is set, or without argument when a flag (`allowed: Type::NONE`) is found. Symbol: instance method; lambda: executed on the plugin instance (same style rule as actions); other: object responding to `call`, e.g. `Log.instance.method(:level=)`. `shorthand:` for a `Hash` option: a `String` value is stored as `{shorthand => value}` (e.g. `--transfer=node`). `deprecation: {last: '4.27.0', message: 'use --out.level'}`: `last` is the last released version supporting the option without deprecation (the next version is not known when coding), shown in the warning, in help (`deprecated after 4.27.0: use --out.level`) and by `config options`
`use_options(source, target:)`	Include options declared by another plugin class or `OptionDeclarator` module. `target:` lambda executed on the plugin instance, returns the object for Symbol and lambda `on_set:` callbacks of `source` (default: the plugin instance), e.g. `use_options Http, target: -> { http_config }`
`application_name(name)`	Human-readable application name shown in wizards

Command declaration (`CommandSpec` attributes):

Parameter	Type	Meaning
`id`	`Symbol`	Unique identifier within its parent's namespace
`parent`	`Symbol | Array<Symbol> | nil`	Full path to parent; `nil` for root commands (usually implied by `commands_under`)
`description`	`String`	User-facing help text
`arguments`	`Array<ArgumentSpec | Hash>`	Positional arguments in parse order. On an **intermediate** node they are resolved before child dispatch (e.g. parent instance id); on a **leaf** they are resolved just before the action
`action`	`Symbol | Proc | nil`	Leaf action. **(1)** omitted → convention `action_<full_path_joined_by_underscores>`; **(2)** `Symbol` → named instance method; **(3)** `Proc` → inline, executed with `instance_exec`
`setup`	`Symbol | nil`	Instance method called with `**ctx` after the node's `arguments` are resolved; returns a `Hash` merged into `ctx` for all descendants
`aliases`	`Array<Symbol> | nil`	Alternative names accepted for this command (e.g. `aliases: [:recv]`), not accepted when the command is excluded by `condition:`
`transfer_paths`	`:send | :receive | nil`	File-list resolution delegated to `TransferAgent` (reads what remains after declared `arguments`)
`condition`	`Symbol | nil`	Instance method returning `Boolean`; if `false`, command is excluded from dispatch but shown in help with an annotation. Not evaluated with `--help`

Parameter	Type	Meaning
`query_schema`	`String | nil`	Schema path for `--query` help; the runner then hints `--query=help`
`mount`	`Hash | nil`	Expose a sub-tree of another plugin class under this node (see Mounting another plugin's commands)

Argument declaration (`ArgumentSpec`, usually given as a `Hash`):

Parameter	Type	Meaning
`name`	`Symbol`	Key in `ctx`, also used in help and error messages
`description`	`String`	User-facing description
`type`	`Class | Array<Class> | :identifier | nil`	Validated type, default `String` (not set when `allowed:` is given); `:identifier` resolves via `options.instance_identifier` (supports percent-selector); explicit `nil` accepts any value (e.g. `config echo`)
`mandatory`	`Boolean`	Default `true`; optional arguments must come after all mandatory ones
`multiple`	`Boolean | String`	`true`: consume all remaining; `String`: consume until the named marker
`default`	`Object | nil`	Default value when `mandatory: false` and no argument provided
`schema`	`String | nil`	JSON schema name for validation and `--help` introspection
`bulk`	`Boolean`	Result is always an `Array`; with `--bulk=yes` the argument is read as an array. Other arguments (e.g. the id of a parent node) are single values with `--bulk=yes`
`lookup`	`Symbol | Proc | nil`	Percent-selector resolver for `:identifier`: `send(lookup, field, value, **ctx)` or `instance_exec(field, value, **ctx, &lookup)`
`allowed`	`Array<Symbol> | nil`	Allowed values (accept list)
`interactive`	`Boolean`	Prompt for the value when missing, for this argument only (`options.with_interactive`): an action that prompts for other values uses `options.with_interactive` explicitly
`complete`	`Symbol | nil`	Values proposed by shell completion, one of `Complete::KINDS`: `:file`, `:folder` (local paths), `:destination_folder` (local folder unless the command sends files), `:preset`, `:preset_parameter`, `:transfer_job`. Also a parameter of `option`, and `x-cli-complete` (`file`, `folder`) for a key of a schema

Arguments already present in `ctx` are not read from the command line: this is how a caller (e.g. a mount seed) or a leaf `setup:` provides a value. If the next argument on the command line is that same value, or `ALL`, it is skipped (e.g. `faspex5 packages receive ALL` with a public link to a package): other arguments are left for the following declared arguments, or for `transfer_paths:`.

Rule: every positional argument is declared with `arguments:`, so that `--help`, completion and `config commands` show it. Plugins never read positional arguments themselves (`options.get_next_argument`, `options.instance_identifier`). When an argument may come from elsewhere, declare it anyway and let the leaf `setup:` inject it in `ctx`, since a leaf setup runs before its arguments are resolved:

```
command :show, description: 'Show a package', setup: :setup_package_id,
  arguments: [{name: :package_id, type: :identifier}],
  action: ->(package_id:, **) { ... }

# With a public link to a package, the id comes from the link and is not read from the command line
def setup_package_id(**)
  return {} unless @api_v5.pub_link_context&.key?('package_id')
  {package_id: @api_v5.pub_link_context['package_id']}
end
```

Static validation: `CommandRegistry#validate!` is run for every plugin by `plugin_registry_spec.rb`, and once per class at first dispatch. It checks that:

- the parent of each `commands_under` is declared
- each leaf has an action (explicit, or method `action_<path>`) accepting any keyword (`**`)
- the methods named by `setup:`, `condition:`, `lookup:` and mount `instance:` exist
- arguments are readable: no mandatory argument after an optional one, no argument after a `multiple: true` one, `lookup:` only with `type: :identifier`, `complete:` one of `Complete::KINDS`
- an alias designates a single command, and does not hide a sibling command
- mounts are consistent (see Mounting another plugin's commands)

The spec also checks that the test command displayed by the wizard of each plugin (`test_args`) is a leaf command of the plugin (`CommandRegistry#command_path`).

`crud_commands` helper:

For each verb in `operations:` (default `Operations::ALL` = `create list modify show delete`) it registers a leaf command whose action calls `entity_<verb>(api:, entity:, **kwargs, **ctx)`:

- `show` / `modify` / `delete` get an `{name: :id, type: :identifier, lookup: lookup}` argument (unless `is_singleton:`); `delete` is bulk-capable
- `create` / `modify` get a `data` `Hash` argument; with `body_component:` its schema is taken from the OpenAPI registry
- `api:` is resolved at runtime: `:@ivar` → instance variable, other `Symbol` → method call, `Proc` → `instance_exec`
- `entity:` may be a `Symbol`, resolved at runtime from `ctx` (e.g. `:sf_entity` injected by a parent `setup:`)
- other `**kwargs` (`display_fields:`, `items_key:`, `is_singleton:`, ...) are forwarded to every `entity_<verb>`

```
commands_under :workflows do
  crud_commands api: :@automation_api, entity: 'workflows'
end
```

Dispatcher algorithm (`Base#execute_action` → `dispatch_from_registry`):

`execute_action` validates the registry once per class (`CommandRegistry#validate!`), then calls `dispatch_from_registry([])`.

```
dispatch_from_registry(current_path, ctx = {})
  spec    = registry[current_path]
  is_leaf = spec has no children

  # Phase A (skipped when --help)
  if !is_leaf
    resolve spec.arguments into ctx          # e.g. parent instance id (:identifier + lookup)
  ctx = ctx.merge(send(spec.setup, **ctx)) if spec.setup

  # Phase B
  if is_leaf
    dispatch_leaf:  raise HelpRequest if --help
                    execute_leaf: resolve spec.arguments into ctx, invoke action_for(spec) with **ctx
  else
    dispatch_child: raise HelpRequest if --help and no more args
                    command = options.get_next_command(children not hidden by condition, aliases:)
                    raise HelpRequest if --help and no more args
                    if child is mounted and not --help
                      return dispatch_mount(...)   # continue on the target plugin instance
                    dispatch_from_registry(current_path + [command], ctx)
```

Key properties of the `ctx` hash:

- **Additive**: each resolved argument and each `setup:` call enriches the accumulated context; descendants can rely on everything resolved by ancestors.
- **Intermediate node**: arguments and setup are resolved before the child command is selected, so context is available to all children.
- **Leaf node**: setup runs, then arguments are resolved, then the action receives `ctx` as keyword arguments.
- `transfer_paths:` — `TransferAgent#ts_source_paths` reads the file list from what remains in the argument stream, depending on `--sources`.

3.2.6 Action style convention

Body size is counted in lines (excluding `do`/`end` or `def`/`end`):

Action body	Form
1 line	inline `command :x, …, action: ->(**){…}` (or `->(arg:, **){…}`), without parentheses on `command`
2–3 lines	inline `command(:x, …, action: lambda do |**| … end)`, with parentheses on `command`
4 lines or more	named method `def action_<full_path>(**)`, no `action:`
used by several commands	named method regardless of size

Parentheses on `command` are only needed for `lambda do…end` (see precedence rule below), and when rubocop requires them: a call inside a method body (e.g. `command(...)` in a DSL class method such as `sync_admin_commands`) is not a macro.

Exception: commands declared by a loop or from a table (e.g. `COMMANDS_GEN4_SPEC`, `define_action_method` in an `each`) have no `command(...)` call of their own to hold an inline action: they use a named method (or a block shared by the loop) regardless of size.

An action always accepts any keyword (`**`): it receives the whole accumulated context, whose keys depend on setups and arguments of ancestors. `validate!` rejects a lambda or method without `**` (a non-lambda `proc` is not checked, as it ignores extra keywords).

The same convention applies to `lookup:`.

Precedence rule: `lambda do...end` has low binding priority — if `command` is called without parentheses, Ruby attaches the `do...end` to `command` instead of `lambda`, causing `tried to create Proc object without a block` at class load time. Always use `command(...)` with parentheses when the action is a `lambda do...end`.

Do not use `lambda{ }` (braces with `lambda` keyword): rubocop's `SpaceInsideBlockBraces` forbids inner spaces, making multi-line bodies unreadable. The codebase uses `->{}` for one-liners and `lambda do...end` for multi-line — no other form.

Examples:

```ruby
# 1 line
command :info, description: 'Show node info',
  action: ->(**){Result::SingleObject.new(@api_node.read('info'))}

command :show, description: 'Show a package',
  arguments: [{name: :package_id, type: :identifier}],
  action: ->(package_id:, **){Result::SingleObject.new(@api.read("packages/#{package_id}"))}

# 2-3 lines: parentheses are mandatory
command(
  :flush, description: 'Delete all cached OAuth tokens',
  action: lambda do |**|
    require 'aspera/api/node'
    Result::ValueList.new(OAuth::Factory.instance.flush_tokens, name: 'file')
  end
)

# 4 lines or more: implicit named method
commands_under :package do
  command :receive, description: 'Receive a package'
end

def action_package_receive(**)
  # many lines of logic...
end
```

All forms receive the `ctx` hash as keyword arguments and behave identically at runtime.

Notable design decisions:

Decision	Rationale
Flat registry keyed by full path `Array<Symbol>`	Avoids recursive data structures; path lookup is O(1)
`arguments:` on intermediate nodes	Parent instance ids are declared statically, so help shows them and dispatch resolves them before child selection
`setup:` runs on the current node before dispatching	Derived context (entity paths, sub-tree specific API or scope, e.g. `aoc automation`, `preview scan`) computed once for the whole sub-tree; no virtual nodes needed
Main plugin API built by a lazy memoized accessor (e.g. `aoc_api`, `api_v5`, `api_node`), not in `initialize` nor by a `setup:` on each root command	Built on first use only: `--help` and `config commands` never need the URL or credentials; commands without authentication (e.g. `faspex5 health`) just do not call it
`condition:` commands visible in help but excluded at runtime	Static documentation is complete; runtime filtering via a method
`crud_commands` generates one leaf per verb	CRUD declarations stay DRY while each verb remains a real, introspectable `CommandSpec`; `api:` is resolved at runtime so the API object can be created lazily
`mount:` instead of re-declaring another plugin's commands	The target sub-tree is declared once: dispatch, `--help`, completion and `config commands` see it entirely, and changes in the target need no change in the hosts
`transfer_paths: :send|:receive`	The `--sources` mechanism in `TransferAgent` cannot be expressed as static arguments
`define_action_method` for homogeneous command groups	Avoids repetitive action definitions for commands sharing the same body (e.g. `ADMIN_OBJECTS` in `aoc.rb`, `RESOURCE_CONFIG` in `faspex5.rb`)

3.2.7 Entity identifier placement

Convention: The CLI uses two distinct argument-order patterns depending on the depth of the command tree:

Pattern	Argument order	When to use
Leaf CRUD	`<entity> <verb> <id>`	The id identifies the *target* of a terminal verb (`show`, `delete`, `modify`). The id comes **after** the verb.
Intermediate selector	`<entity> <sub-tree> <id> <verb>`	The id selects a *parent instance* whose children will be further dispatched. The id comes **before** the sub-tree verb.

The second pattern is declared with an `:identifier` argument **on the intermediate node** — never read inside the leaf actions. If children need derived values (entity path, API object), add a `setup:` on the same node; it runs after the argument is resolved.

Wrong — identifier for an intermediate node read in the leaf action (too late):

```
# BAD: forces the user to write: workspace dropbox list <id>
# but the convention requires:  workspace dropbox <id> list
def action_admin_workspace_dropbox_list(**)
  ws_id = options.instance_identifier   # ← consumed AFTER command selection
  ...
end
```

Correct — identifier declared on the intermediate node, derived context from `setup:` (from `faspex5.rb`):

```
commands_under %i[admin nodes] do
  command :shared_folders, description: 'Manage shared folders',
    arguments: [{name: :node_id, type: :identifier, lookup: :lookup_node_id}],
    setup: :setup_admin_nodes_shared_folders
end
```

```ruby
commands_under %i[admin nodes shared_folders] do
  crud_commands entity: :sf_entity, api: :@api_v5, name: 'shared folder',
    lookup: :lookup_sf_id, items_key: 'shared_folders'
end

# node_id: already in ctx from arguments: on :shared_folders
def setup_admin_nodes_shared_folders(node_id:, **)
  {sf_entity: "nodes/#{node_id}/shared_folders"}
end
```

Quick test: if the id selects *which* sub-tree to enter (further commands follow), it belongs in the intermediate node's `arguments:`. If the id identifies the *object* of the terminal operation (nothing follows), it belongs in the leaf's `arguments:`.

Examples in the codebase:

Node	Argument	Setup → injects into ctx
`aoc admin workspace dropbox`	`workspace_id`	`setup_admin_workspace_dropbox` → `ws_res_id:`
`aoc admin workspace shared_folder`	`workspace_id`	`setup_admin_workspace_shared_folder` → `ws_res_id:`, `shared_folders:`
`faspex5 admin nodes shared_folders`	`node_id`	`setup_admin_nodes_shared_folders` → `sf_entity:`
`faspex5 admin nodes shared_folders user`	`sf_id`	`setup_admin_nodes_shared_folders_user` → `user_path:`
`node access_keys do`	`access_key_id`	`setup_access_key_do` → `do_root_file_id:`
`node access_keys do <id> permission`	`path`	`setup_access_key_do_permission` → `apifid:`
`aoc packages shared_inboxes short_link`	`link_type`, `dropbox_id`	`setup_packages_short_link` → `shared_data:`, …

3.2.8 Mounting another plugin's commands

Several plugins expose commands of another plugin, typically `Node` (COS bucket, AoC files, ATS access key, Shares files). The `mount:` attribute of a command node exposes the children of a node of another plugin class, instead of re-declaring them:

```ruby
command :node, description: 'Execute COS node commands',
  mount: {plugin: Node, instance: :cos_node_plugin, only: Node::COMMANDS_COS}

def cos_node_plugin(**)
  Node.new(context: context, api: Api::CosNode.new(...))
end
```

Key	Meaning
`plugin`	Target plugin class
`at`	Path in the target registry whose children are mounted (default `[]`: root)
`instance`	Host method called with `**ctx` (after the node's own `arguments:` and `setup:`), returning the target instance, or `[instance, seed_ctx]`
`only` / `except`	Filter the mounted children
`arguments`	Arguments read by the host right after the mounted command, before the target's own arguments, and passed to `instance` in `ctx` (mandatory only)

Semantics:

- **Registry**: `CommandRegistry#[]`, `children_of`, `leaf_paths` follow mounts, with paths in the host namespace. Host children declared under the mount node are merged with the mounted ones and take precedence on an id conflict.

- **Dispatch**: when a mounted child is selected, `dispatch_mount` resolves the mount `arguments`, calls `instance`, then `target.dispatch_from_registry(at + [command], seed_ctx)`. Arguments and setups **below** the mount point run normally in the target; those of `at` and its ancestors do not run: `seed_ctx` provides what they would have put in `ctx` (e.g. `do_root_file_id:` for `at: %i[access_keys do]`).
- **Arguments**: `CommandRegistry#arguments_at(path)` returns the mount `arguments` followed by the node's own, e.g. `aoc packages ls <package_id> <path>`: `--help` and `config commands` use it.
- **Help**: with `--help`, the host keeps walking its mount-aware registry: the target is never instantiated (no API connection), and `condition:` of mounted commands is not evaluated.
- **Validation**: `validate!` checks that `instance` exists, that `at` and the `only`/`except` ids exist in the target, that the mount node has no `action:`, and that mount `arguments` are mandatory.

Mount points:

Host	Target
`cos node`	`Node` root, `only: COMMANDS_COS`
`shares files`	`Node` root, `only: COMMANDS_SHARES`
`ats access_key node <id>`	`Node` `access_keys do`
`aoc files`, `aoc admin node do <id>`, `aoc admin workspace shared_folder <id> node <id>`	`Node` `access_keys do` (plus the AoC-specific `transfer`, and `short_link` for `files`)
`aoc packages`	`Node` `access_keys do`, `only: NODE4_READ_ACTIONS`, `arguments: <package_id>`
`aoc admin ats`	`Ats` root
`node access_keys do <id> v3`	`Node` root (a mount on itself: `leaf_paths` does not expand a cycle twice)

`mount:` is the only cross-plugin delegation mechanism of the DSL.

3.2.9 Legitimate residual imperative reads

Plugin-level imperative dispatch and argument reads have been eliminated. The remaining occurrences are infrastructure and are intentional:

File	Location	Role
parser.rb	`get_next_command`	Infrastructure — defines `get_next_command`
base.rb	`dispatch_child`	Infrastructure — the DSL dispatcher itself calls `get_next_command`
runner.rb	`run_with_result`	Top-level plugin selector (`case command_sym`), not a per-plugin dispatch
base.rb	`dispatch_from_registry`, `execute_leaf`, `resolve_argument`	Infrastructure — resolution of declared `arguments:`
transfer_agent.rb	`ts_source_paths`	Infrastructure — source file list of `transfer_paths:` commands (depends on `--sources`)
preset_actions.rb	`action_preset_ask`	`options.get_interactive(option_name)`: option names are only known at runtime, cannot be declared as interactive arguments

No plugin file uses `get_next_command`, `get_next_argument` or `instance_identifier`.

3.2.10 Transfer Agent Abstraction

File: lib/aspera/cli/transfer_agent.rb

The Transfer Agent provides a unified interface for initiating transfers across different FASP clients:

Responsibilities:

- Abstract transfer initiation across multiple agent types
- Manage transfer specifications (transfer_spec)
- Handle file list sources (`@args`, `@ts`, arrays)
- Coordinate transfer progress monitoring (`transfer_progress.rb`)
- Send transfer completion notifications
- Track asynchronous transfers: jobs are persisted by `job_id` in `async_transfer_store.rb` and can be re-queried later

Agent Base Class (`lib/aspera/agent/base.rb`):

```
class Base
  # Optional: re-query a previously started transfer by id (desktop, node, connect, transferd, ...)
  def self.transfer_status(transfer_id, agent_params)

  # Not defined in Base, each subclass must implement:
  #   start_transfer(transfer_spec, token_regenerator: nil)  start a transfer asynchronously
  #   wait_for_transfers_completion                          wait and return per-session statuses

  # Wait for completion; returns Transfer::Result::Success or Transfer::Result::Error (public API)
  def wait_for_completion

  # Job id of the last submitted transfer (in-process agents)
  def last_job_id

  # Optional: release resources
  def shutdown
end
```

Transfer outcomes are typed (`transfer/result.rb`): `Transfer::Result::Success`, `Transfer::Result::Error`, `Transfer::Result::Async`.

Supported Agents (`lib/aspera/agent/`, created by `Agent::Factory`):

- **Direct**: Direct `ascp` execution (default)
- **Connect**: Aspera Connect browser plugin
- **Node**: Node API-based transfers
- **Desktop**: Aspera Desktop Client, which runs the Transfer SDK underneath
- **Transfer Daemon (trSDK)**: gRPC-based transfer service (`transferd.rb`)
- **HTTPGW**: HTTP Gateway, transfers over HTTPS and WebSocket (`httpgw.rb`)

3.3 API Communication Layer

3.3.1 REST Client

Folder: `lib/aspera/rest/` (all loaded by `lib/aspera/rest.rb`)

File	Content
`client.rb`	`Rest::Client`: the HTTP client
`parameters.rb`	`Rest::Parameters`: global settings (singleton)
`util.rb`	`Rest.build_uri`, `Rest.php_style`, `Rest.query_to_h`, `Rest.parse_header`, `Rest.basic_authorization`, `Rest.start_http_session`, ...
`call_error.rb`	`Rest::CallError`: raised on HTTP call errors
`error_analyzer.rb`	`Rest::ErrorAnalyzer`: extracts error messages from responses (singleton)
`aspera_errors.rb`	Error handlers for Aspera APIs, registered on require
`list.rb`	`Rest::List`: paginated listing and lookup, included in API clients

`Rest::Client` is a custom HTTP client implementation providing:

- **HTTP Methods**: GET, POST, PUT, PATCH, DELETE, CANCEL
- **Authentication**: Basic, Bearer token, OAuth 2.0
- **Content Types**: JSON, form-encoded, multipart

- **Error Handling**: Automatic retry logic, error analysis
- **Progress Tracking**: File upload/download progress
- **Session Management**: Connection pooling, SSL/TLS configuration, proxy auto-config (`proxy_auto_config.rb`)

Global HTTP settings are held in the `Rest::Parameters` singleton. The CLI HTTP/S and TLS options (`insecure`, `ignore_certificate`, `cert_stores`, `http_options`, `http_proxy`, ...) are declared and applied by `Cli::Http` (`Context#http_config`). Paginated listing is handled by `Rest::List`.

3.3.2 Product API Clients

Directory: `lib/aspera/api/` — `aoc`, `ats`, `alee`, `cos_node`, `faspex`, `httpgw`, `node`

Node API Client (`node.rb`):

- **Access Key Management**: Gen4 access key support
- **Bearer Token Generation**: JWT-based authentication
- **File Operations**: Browse, upload, download, delete
- **Permission Management**: Fine-grained access control
- **Transfer Spec Generation**: Automatic transfer parameter creation
- **Cache control**: optional request header to bypass the server-side (Redis) cache

3.3.3 API Schemas

Directory: `lib/aspera/schema/`

OpenAPI definitions of the product APIs (AoC, AoC Automation, Faspex 5, Node, Shares, Console, faspio), `async` database tables (`async_tables.yaml`) and `Schema::Registry`, used to:

- document request bodies of `create` / `modify` (`body_component:` in `crud_commands`)
- document `--query` parameters (`query_schema:`, shown with `--query=help`)

`Schema::Validator` validates values of options and arguments against their `schema:` (gem `json_schemer`), only for schemas owned by ascli (`Registry::OWNED`: options, transfer spec, sync, async tables): vendor API schemas are validated by the API. Option values are partial (merged from presets and command line): `required` is not enforced, and a `oneOf` whose `discriminator` property is absent is not checked.

3.3.4 OAuth Implementation

Directory: `lib/aspera/oauth/`

Modular OAuth 2.0 support, instantiated through `OAuth::Factory` (which also caches tokens):

- **Generic OAuth** (`generic.rb`): Standard OAuth 2.0 flows
- **JWT** (`jwt.rb`): JSON Web Token authentication
- **Web** (`web.rb`): Browser-based OAuth flows; the local callback server lives in `lib/aspera/web_auth/`
- **URL JSON** (`url_json.rb`): Token from URL
- **Boot** (`boot.rb`)

3.4 FASP Transfer Layer

3.4.1 ASCP Installation Manager

File: `lib/aspera/ascp/installation.rb`

Singleton class managing `ascp` binary location and SDK resources:

- **Product Detection**: Automatically finds installed Aspera products (`lib/aspera/products/`)
- **SDK Installation**: Downloads and installs Transfer SDK
- **Path Resolution**: Locates `ascp` executable and supporting files
- **SSH Key Management**: Handles client SSH keys for authentication

Supported product detection:

- Aspera Desktop Client
- Aspera Connect
- Aspera Transfer SDK (`transferd`)
- Aspera HSTS/ATS installations

3.4.2 Transfer Specification

File: `lib/aspera/transfer/spec.rb` (schema: `spec.schema.yaml`)

Transfer specifications define all parameters for a FASP transfer:

- Source and destination paths
- Transfer direction (upload/download)
- Rate control (target rate, min rate, policy)
- Encryption settings
- Resume policies
- Authentication credentials
- Protocol options (UDP/TCP ports, SSH options)

3.4.3 Async Sync

Directory: `lib/aspera/sync/` — `async` (Aspera Sync) arguments, configuration schemas and database access, used by `SyncActions`.

3.5 Remote Systems Layer

The CLI communicates with various IBM Aspera components:

3.5.1 Web Applications (HTTPS)

- **Aspera on Cloud (AoC)**: Cloud-based file sharing and collaboration
- **Aspera Transfer Service (ATS)**: Managed transfer service
- **Faspex 5**: Secure package exchange
- **Shares**: File sharing and synchronization
- **Console**: Central management console
- **Orchestrator**: Workflow automation
- **faspio Gateway**: FASP acceleration of TCP traffic
- **Aspera License Entitlement Engine (ALEE)**: Entitlement and metering
- **HTTP Gateway**: Transfers over HTTPS and WebSocket, the gateway transfers with FASP to the transfer server (agent `httpgw`)

Communication via:

- REST APIs over HTTPS
- OAuth 2.0 authentication
- JSON request/response payloads

3.5.2 Transfer Servers (FASP Protocol)

- **IBM Cloud Object Storage (COS)**: S3-compatible object storage with FASP
- **Aspera Transfer Server (ATS)**: Dedicated transfer endpoints
- **HSTS Node**: High-Speed Transfer Server with Node API

Communication via:

- FASP protocol (TCP/UDP) for data transfer
- Node API (HTTPS) for control operations, and transfers started on the server (agent `node`)
- SSH for authentication and session management (and `ascmd` file operations on HSTS, plugin `server`)

3.5.3 Third-Party Integrations

- **gRPC**: Transfer Daemon communication
- **JSON-RPC**: Desktop client communication (`lib/aspera/json_rpc/`)
- **MCP**: Model Context Protocol for AI assistant integration
- **External Tools**: Integration with system utilities (e.g. ffmpeg, ImageMagick, LibreOffice for previews)

Chapter 4

Data Flow

4.1 Typical Command Execution Flow

1. **Command Parsing**:

   ```
   User Input &rarr; bin/ascli &rarr; Runner &rarr; Parser (options + positional args)
   ```

2. **Plugin Selection**:

   ```
   Command &rarr; Plugin Factory &rarr; Specific Plugin (e.g., aoc, faspex5)
   ```

3. **Command Dispatch**:

   ```
   Plugin#execute_action &rarr; dispatch_from_registry &rarr; action (method or Proc)
   ```

4. **API Communication**:

   ```
   Action &rarr; REST Client &rarr; Remote API &rarr; JSON Response
   ```

5. **Transfer Initiation**:

   ```
   Action &rarr; Transfer Agent &rarr; Agent Selection &rarr; ascp/trSDK/Connect/...
   ```

6. **Transfer Execution**:

   ```
   Transfer Agent &rarr; FASP Protocol &rarr; Remote Server &rarr; Progress Updates
   ```

7. **Result Formatting**:

   ```
   Result object &rarr; Formatter &rarr; Output (table/json/yaml/csv/...)
   ```

Chapter 5

Key Design Patterns

5.1 Plugin Architecture

Each Aspera product is implemented as a plugin inheriting from `Plugins::Base`:

- Consistent command structure across products
- Standard CRUD operations
- Extensible for product-specific features

5.2 Factory Pattern

Used for creating instances based on configuration:

- **Agent Factory**: Selects appropriate transfer agent
- **OAuth Factory**: Creates authentication handlers
- **Plugin Factory**: Instantiates product plugins
- **Secret Store Factory**: Selects the secret storage backend

5.3 Singleton Pattern

Used for global configuration and state:

- **Installation**: ASCP binary location
- **Rest::Parameters**: HTTP client settings
- **Log**: Logging configuration

5.4 Strategy Pattern

Transfer agents implement a common interface with different strategies:

- Direct execution via `ascp`
- Browser-based via Connect
- API-based via Node
- Gateway-based via HTTPGW

5.5 Command DSL Pattern

Each plugin declares its command tree at class level (see Command DSL); the base class dispatcher (`dispatch_from_registry`) traverses the registry and calls the appropriate action on the plugin instance.

5.6 Mixin / Module Pattern

Large classes are decomposed into focused mixins included by the host class:

- `Config` plugin includes `SyncActions`, `VaultActions`, `GemChecker`, `AscpActions`, `PresetActions`, `TransferActions`
- Each mixin owns a single responsibility and depends on `options`, `context`, and other accessors provided by the host
- A mixin that declares commands adds class-level DSL methods with `extend` in its `included` hook (e.g. `SyncActions::ClassMethods#sync_admin_commands`), called inside `commands_under` like the other DSL methods

Chapter 6

Configuration Management

6.1 Configuration File

Location: `~/.aspera/ascli/config.yaml`

Stores:

- Preset configurations for different environments
- Default options and parameters
- Authentication credentials (or references to the vault)
- Transfer agent preferences

6.2 Preset System

Presets allow saving commonly used option combinations (`preset_manager.rb`, `preset_actions.rb`):

```yaml
presets:
  my_aoc:
    url: https://mycompany.ibmaspera.com
    username: user@example.com
    password: "@vault:aoc_password"
```

- `config preset set` accepts dot-notation keys (deep-merged, with type coercion)
- Keys starting with `_` (e.g. `_comment`) are ignored
- **Option changes and migrations**: when an option syntax changes and an automatic conversion is possible (e.g. deprecated option format), `PresetManager#validate_config_presets!` inspects the presets, converts them to the new syntax, creates a backup copy of the original file (`<config_file>.pre<VERSION>.bak`), and immediately saves the rewritten configuration file to disk.

6.3 Secret Management

Directory: `lib/aspera/secret/`, module `Secret`: hiding of secrets in output (`Hider`), strength of secrets (`Strength`), stores of secrets (`Store`), secret references (`Reference`, `Resolver`).

Stores (`secret/store/`): subclasses of `Secret::Store::Base`, created by `Secret::Store::Factory.create` from parameters: `type`, and the keyword arguments of `new` of the class of type (`PARAMETERS`, converted by `arguments`). Each store is used in two ways: management by label (`get`, `set`, `delete`, `all`), and read by reference (`check_reference`, `read_reference`: by default, the path is the label and the field is `password`).

- **file**: `EncryptedHash`, `file` and `password`; the file is read when used, and again when it, or the file of the password, is modified
- **system**: `MacosSystem`, macOS Keychain
- **secret_service**: `SecretService`, Linux Secret Service (GNOME Keyring, KWallet...), with `secret-tool`; the password is sent on stdin

- **vault**: `HashicorpVault`, authentication by `token` or `approle`, own `Vault::Client`; labels in mount `secret`, references in any KV v2 mount (version), limited by `allowed_prefixes`; `error_reason` gives the reason of errors of gem `vault`
- **1password**: `OnePasswordApi` or `OnePasswordCli`, selected by `source`
- Types added by the application: `Factory.register_type` (a subclass of `Store::Base` with `PARAMETERS`)

Secret references (`secret/reference.rb`, `secret/resolver.rb`): library for applications that store a reference instead of a secret (e.g. Orchestrator), and `@vault:` of `ascli`.

- `Secret.parse_reference`: `secret://<store>/<path>[?<options>][#<field>]` into a `Reference`
- `Secret::Resolver`: named stores from a configuration (`Hash`: `cache_ttl`, `stores`): type and names of parameters checked by `new` (`Store::Factory.check`), store created when first used (`Store::Factory.create`), so that a store not used is not an error (e.g. its secret is not available); `store` (by name), `resolve` (value, TTL cache), `metadata` (version, time, no value), `fields` (field names of a secret, no value: `read_fields` of the store), `list` (secrets and folders that references can read, without reading them: `list_folder` of the store, labels by default, folders of a KV v2 mount for `vault`, limited by `allowed_prefixes`), `invalidate`; errors give the reference and the reason, never a value; values are declared to `Secret::Hider`
- Bootstrap secrets of stores (`password` of `file`, `token` and `approle.secret_id` of `vault`): `Secret::Bootstrap`, the value or `@file:<path>` (syntax of extended values, only `@file:`), the file read again when modified; `file` converts `password` in `arguments`, `vault` in its constructor

Vault of `ascli` (`vault_manager.rb`, `vault_actions.rb`): `VaultManager` in `Context#vault`, created by `Bootstrapper`, used by `@vault:`, `config vault` (`VaultActions`) and presets (`PresetActions`):

- One `Secret::Resolver` from option `vault`, created when first used (kept once created, not when absent: the global preset is loaded after the handler is registered): `stores` and `cache_ttl`, and `default`, the name of the default store (none if not set)
  - Deprecated form: the parameters of one store, converted by `VaultManager.convert_option` (`convert:` of the option, applied to each value before merge of sources) to a store named after its type (`file` by default), which is the default store, with a warning
  - Defaults of `ascli` for all stores (`store_parameters`): type `file`: `name` (same as `file`), `vault.bin`, relative path in the configuration folder; types `system` (keychain) and `secret_service` (group of secrets): name `ascli`
  - Secret of the default store, if not in its parameters (`password` of `file`, `token` of `vault` without `approle`, and of `1password` `api`): the value of option `vault_password`
- `store`: the default store (`nil` if none)
- `value`: `@vault:<label>.<field>` from the default store, `@vault://<store>/<path>[?<options>][#<field>]` from any store
- `metadata`: `config vault check`, version and time of a reference, without value
- Values `@file:<path>` of options `vault` and `vault_password` are read by `Secret::Bootstrap`, not by `ExtendedValue`: checked as extended value `@file:` (`Security#check_extended_value`, `Security#readable_path`) before the resolver is created

Chapter 7

Error Handling

7.1 Error Hierarchy

```
StandardError
├── Aspera::Error (lib/aspera/assert.rb)
│   ├── Aspera::ParameterError
│   ├── Aspera::InternalError
│   ├── Aspera::AssertError
│   ├── Aspera::EntityNotFound (resource not found — lib/aspera/assert.rb)
│   └── Aspera::Ssh::Error
├── Aspera::Cli::Error (CLI base — lib/aspera/cli/error.rb)
│   ├── BadArgument
│   ├── MissingArgument
│   ├── NoSuchElement
│   ├── BadIdentifier
│   └── SchemaRequest (control flow: `help` given as option or argument value —
↪  lib/aspera/cli/option_types.rb)
├── Aspera::Cli::HelpRequest (control flow: --help reached in dispatch)
├── Aspera::Rest::CallError (HTTP call errors — lib/aspera/rest/call_error.rb)
├── Aspera::Ascmd::Error (ascmd errors — lib/aspera/ascmd.rb)
└── Aspera::Transfer::Error (transfer failures — lib/aspera/transfer/error.rb)
```

7.2 Error Analysis

File: lib/aspera/rest/error_analyzer.rb

Analyzes API errors and provides:

- Human-readable error messages
- Suggested remediation steps
- Context-specific guidance

Chapter 8

Logging and Debugging

8.1 Log Levels

- `trace2`: Finest-grained tracing (most verbose)
- `trace1`: Fine-grained tracing
- `debug`: Detailed debugging information
- `info`: General informational messages
- `warn`: Warning messages
- `error`: Error messages
- `fatal`: Fatal errors
- `unknown`: Unknown severity

8.2 Debug Features

- HTTP request/response logging
- Transfer specification display
- API call tracing
- Progress monitoring
- Secrets are masked in logs (`secret/hider.rb`)

Chapter 9

Testing Architecture

9.1 Test Structure

Directory: `spec/`

- Unit tests, fully self-contained, e.g.:
  - `base_dsl_spec.rb` — `Base` DSL dispatcher
  - `command_registry_spec.rb` — `CommandRegistry` validation rules
  - `plugin_registry_spec.rb` — `validate!(plugin_class:)` consistency of every plugin registry (every leaf has an action or a matching `action_*` method, and every action accepts `**`)
  - `parser_spec.rb`, `option_declarator_spec.rb`, `preset_actions_spec.rb`, `vault_manager_spec.rb`, `runner_spec.rb`, `mcp/tool_spec.rb`, `formatter_spec.rb`, `http_spec.rb`
  - `async_transfer_store_spec.rb`, `transfer_agent_async_spec.rb`, `transfer_agent_options_spec.rb`, `transfer_result_spec.rb`, `agent_transfer_status_spec.rb`
  - `secret/store_spec.rb`, `secret/store/factory_spec.rb`, `secret/reference_spec.rb`, `secret/reference_vault_spec.rb` (fake Vault server), `secret/reference_encrypted_file_spec.rb`, `secret/strength_spec.rb`
  - `rest/client_spec.rb`, `secret/hider_spec.rb`, `proxy_auto_config_spec.rb`, `schema_reader_spec.rb`, `string_ext_spec.rb`, `uri_reader_spec.rb`, `temp_file_manager_spec.rb`, `assert_spec.rb`, `environment_spec.rb`, `version_spec.rb`, ...
- Live tests requiring a server, tagged `:live` (e.g. `ascmd_ssh_live_spec.rb`)

9.2 CI/CD Integration

GitHub Actions workflows (`.github/workflows/`):

- Multi-version Ruby testing, and JRuby (matrix in `test.yml`)
- Code quality checks (RuboCop)
- Security scanning (CodeQL)
- Release, deploy, certificate renewal

Chapter 10

Extension Points

10.1 Adding a New Plugin

1. Create plugin file in `lib/aspera/cli/plugins/` (or in a plugin lookup folder)
2. Inherit from `Plugins::Base` (or `BasicAuth` / `Oauth`)
3. Declare options with `option(...)` and commands with `command(...)` / `commands_under` / `crud_commands` at class level
4. Implement `action_<path>` methods for leaf commands without an inline `action:`

The plugin factory discovers the plugin automatically; `CommandRegistry#validate!` checks the tree at first execution.

10.2 Adding a New Transfer Agent

1. Create agent file in `lib/aspera/agent/`
2. Inherit from `Agent::Base`
3. Implement required methods:
   - `start_transfer`
   - `wait_for_transfers_completion`
   - optionally `self.transfer_status` (async support) and `shutdown`
4. Register in `Agent::Factory`

10.3 Adding a New Output Format

1. Add the format to `DISPLAY_FORMATS` in `formatter.rb` (allowed values of option `format`)
2. Implement the format-specific rendering in `Formatter#display_results`

Chapter 11

Performance Considerations

11.1 Transfer Optimization

- **Multi-session**: Parallel transfer sessions for large files
- **Adaptive Rate**: Dynamic bandwidth adjustment
- **Resume**: Sparse checksum-based resume
- **Compression**: Optional in-flight compression

11.2 API Optimization

- **Pagination**: Efficient handling of large result sets
- **Token Caching**: OAuth tokens cached on disk
- **Connection Pooling**: Reuse HTTP connections
- **Batch Operations**: Bulk create/delete operations

Chapter 12

Security Architecture

12.1 Authentication Methods

1. **OAuth 2.0**: Token-based authentication
2. **JWT**: JSON Web Tokens
3. **Basic Auth**: Username/password
4. **SSH Keys**: Public key authentication
5. **Access Keys**: Node API access keys

12.2 Credential Storage

- Encrypted configuration file
- System keychain integration
- Environment variables
- Vault integration (HashiCorp Vault, 1Password)

12.3 Secure Communication

- TLS/SSL for HTTPS
- SSH for FASP control channel
- Encrypted FASP data transfer
- Certificate validation

Chapter 13

Deployment Models

13.1 Installation Methods

1. **Ruby Gem**: `gem install aspera-cli`
2. **Single Executable**: Standalone binary
3. **Windows portable package**: ZIP archive with Ruby, gems and Transfer SDK (extract and run, no installation)
4. **Windows installer**
5. **Container**: Docker image
6. **Package Managers**: Chocolatey

13.2 Runtime Requirements

- Ruby, see `RUBY_MINIMUM_VERSION` in `lib/aspera/cli/info.rb`
- FASP client (ascp or Transfer SDK)
- Network connectivity
- Sufficient disk space for transfers

Chapter 14

References

- Main Documentation
- Contributing Guide
- API Documentation
- IBM Aspera Documentation

End of document